Optimistic Rollups:
Scaling Ethereum Without Sacrificing Security

Final Report — Topic 13a

Vaasu Akhil
(equal contribution)

CS 521 — Course Project — Final Submission

Abstract

Optimistic rollups execute transactions off-chain and commit only a state-root and transaction data to Ethereum L1, relying on a *challenge window* and *interactive fraud proofs* to guarantee correctness without per-batch validity proofs. This report synthesizes optimistic rollup design—architecture, bisection fraud-proof mechanics, Arbitrum's BoLD versus Optimism's Cannon, EIP-4844 data availability, and comparisons with ZK rollups—and presents our working implementation: `InteractiveOptimisticRollup`, a Solidity contract with TypeScript runner scripts deployed to Base Sepolia testnet. We use an integer-delta state model to exhibit the full dispute lifecycle end-to-end, with all on-chain transaction hashes and gas costs reported from a live sequencer-fraud demonstration, and discuss explicitly what Merkle-committed state would require differently at each bisection midpoint and at single-step verification.

1. Introduction

Layer-2 scaling is no longer a research direction; it is the production reality of Ethereum. As of 2026, the majority of user-facing on-chain activity does not occur on Ethereum L1 but on rollups that inherit L1's security while operating their own execution environment. Among rollup constructions, *optimistic rollups*—first deployed by Arbitrum and Optimism—currently host the deepest pools of liquidity and the largest installed base of EVM-equivalent applications.

The intuition behind an optimistic rollup is captured in one phrase: **innocent until proven guilty**. Rather than requiring every transaction to be cryptographically proven correct before acceptance (the approach taken by zero-knowledge rollups), the system *assumes* that the rollup operator's claimed state transitions are valid and only intervenes if some independent party submits evidence to the contrary within a fixed dispute window. This "optimism" is what enables optimistic rollups to be cheap and EVM-compatible—there is no per-batch SNARK to compute—but it is also why the design space for fraud proofs, sequencer accountability, challenge timing, and data availability is so rich. Every parameter trades scalability against security, finality against responsiveness, and decentralization against operator efficiency.

2. The Problem: Ethereum Layer 1 Does Not Scale

Ethereum's execution layer was not designed for application-layer throughput. Every full node re-executes every transaction in every block, which gives L1 its security properties but bounds throughput to roughly 15–30 TPS. The *scaling trilemma* implies that scaling cannot come for free; the strategy adopted by rollups is to give up *self-contained execution*: computation happens off-chain while L1 stores only the data needed to reconstruct state and the cryptographic commitment that pins down what that state is. **L1 only needs to be the source of truth, not the executor of every step.**

3. What Is an Optimistic Rollup?

An *optimistic rollup* is an L2 protocol that:

1. Executes transactions off-chain on its own execution environment (typically EVM-compatible).

2. Compresses and posts the resulting transaction data, plus a *state-root commitment*, to L1 in batches.
3. Treats each posted state root as valid by default—*optimistically*—without proving its correctness up front.
4. Permits any independent observer, during a fixed *challenge window* (industry standard: ∼7 days), to submit a *fraud proof* showing that the posted state root does not correspond to honest execution of the posted transactions.
5. If a fraud proof succeeds, the offending batch (and all subsequent batches that depended on it) is invalidated, and the dishonest sequencer's bond is slashed.
6. If the challenge window closes with no successful fraud proof, the state root is considered final on L1.

Security derives from two assumptions: **data availability** (all transaction data is posted to L1, so anyone can independently reconstruct state) and **at least one honest challenger** (a single non-colluding party can submit a fraud proof when warranted). This is a "1-of-N" security model—anyone can become a challenger by posting the required bond.

3.1 Transaction Lifecycle

Transactions proceed through four phases: (1) user submits, gets soft confirmation in seconds; (2) sequencer batches, executes, posts commitment to L1; (3) challenge window (∼7 days) during which any party may dispute; (4) state root finalized if no challenge succeeds. **Deposits (L1→L2) are fast** (minutes); **withdrawals (L2→L1) wait out the challenge window**. Third-party fast bridges provide same-block liquidity at a fee.

4. Core Architecture

An optimistic rollup is built from four interacting components.

4.1 Sequencer

The sequencer is the rollup's transaction-ordering and execution authority. It receives user transactions, sequences them, executes them against current state, batches them, and posts the resulting commitment to L1. In every major production deployment today the sequencer is **centralized**—a deliberate operational simplification providing better latency and simpler MEV economics, at the cost of a known censorship vector. The mitigation is twofold: the sequencer cannot steal funds (because of fraud proofs), and all rollups expose a *forced inclusion* mechanism allowing users to bypass a misbehaving sequencer via L1 directly.

The sequencer's economic accountability is enforced by a **bond**: collateral posted on L1 that is slashed if any fraud proof against its batches succeeds.

4.2 State Root, Challengers, and L1 Contracts

The **state root** is a 32-byte Merkle root over the rollup's full state after the batch's last transaction—the sequencer claims "root is R"; the challenger asserts "no, it is R′". **Challengers** are permissionless: anyone holding the required bond can replay batches and submit fraud proofs; a wrong proof loses the challenger's bond, while a correct one earns the sequencer's. The **L1 contracts** hold both bonds, store batch headers, implement the dispute state machine, and process deposits/withdrawals—the trust anchor of the entire system.

5. Fraud Proofs: The Core Security Mechanism

A fraud proof is a procedure executed on L1 demonstrating that the sequencer's posted state root is inconsistent with honest execution of the posted transactions. There are two architectural approaches.

5.1 Non-Interactive Fraud Proofs

In a non-interactive scheme, the challenger submits a proof of incorrect execution verifiable in a single transaction. The direct version—L1 re-executes the entire disputed batch—is conceptually clean but operationally infeasible: a typical batch may contain thousands of transactions whose re-execution would consume more gas than the rollup was designed to save.

5.2 Interactive Fraud Proofs (Bisection Games)

The interactive approach, used by all production optimistic rollups today, reduces a disputed batch to a *single execution step* through a binary-search game:

1. The sequencer and challenger each commit to a sequence of intermediate state roots—one per transaction.
2. They disagree on the final root, so there exists a first index of divergence.
3. The L1 contract orchestrates bisection: at each round, the disputed range is halved by examining the midpoint state root.
4. After $O(\log N)$ rounds, the range is a single step.
5. L1 re-executes that one step; the party whose claim matches actual execution wins, the other is slashed.

This means **L1 only ever re-executes one step**, regardless of batch size. The cost of the dispute is logarithmic in transaction count for bisection rounds plus a constant for single-step verification.

5.3 Bonds and Economic Security

Both sides post bonds before participating. On resolution, the loser's bond is forfeit—part awarded to the winner as a bounty (incentivizing honest challenge), part burned or paid to the protocol treasury (preventing griefing collusion). **Fraud is negative-EV for any rational actor whose bond exceeds the extractable value from lying.** No successful mainnet fraud has occurred in either Arbitrum or Optimism.

6. Production Comparison: Arbitrum (BoLD) vs. Optimism (Cannon)

Both systems use interactive fraud proofs, EVM-compatible execution, a single sequencer, a challenge window, and on-chain rollup contracts. The substantive difference is *what virtual machine the single-step re-execution runs on.*

6.1 Arbitrum: BoLD

Arbitrum's dispute protocol is called **BoLD** (Bounded Liquidity Delay)—the rule system governing the bisection game. The single-step re-execution runs against an EVM-equivalent environment: the disputed step is an actual EVM instruction on actual EVM state, with no translation layer. The advantage is directness; the constraint is that Arbitrum's L2 execution must remain EVM-equivalent to make this work.

6.2 Optimism: Cannon

Optimism's fault-proof system uses **Cannon**, a MIPS CPU emulator implemented in Solidity. Optimism's L2 execution client (a fork of go-ethereum) is compiled to MIPS bytecode; a single disputed step is a MIPS instruction executed inside Cannon. The advantage is decoupling—Optimism can upgrade its execution client without changing on-chain dispute infrastructure. The cost is the translation layer: the program must be deterministically compiled to MIPS, and the on-chain interpreter must exactly reproduce its semantics.

Both systems reduce a dispute to verifying a *single execution step* on L1—an EVM opcode in Arbitrum's case, a MIPS instruction in Optimism's—different points on a design surface trading directness against decoupling, not fundamentally different security models.

7. Data Availability and EIP-4844

Data availability is a prerequisite for the security of optimistic rollups. Without it, challengers cannot reconstruct state and cannot submit fraud proofs: if a sequencer posts a batch header but withholds the underlying transaction data, honest challengers are blinded.

The original data-availability solution was to post transaction data as **calldata** on Ethereum L1—permanently accessible, billed at 16 gas per non-zero byte, and dominant in the fees users pay on L2 at peak prices.

7.1 EIP-4844 (Proto-Danksharding)

EIP-4844, activated in the Cancun hard fork (March 2024), introduced **blob-carrying transactions** as a substantially cheaper alternative. Key properties:

- **Large and cheap.** Blobs are ∼128 KB each, priced on a separate fee market (the *blob base fee*) that adjusts independently of EVM gas. Under normal conditions, a blob costs a fraction of equivalent calldata.
- **Finite retention.** Blobs are stored by Ethereum nodes for approximately 18 days, then pruned. This window was chosen to exceed the 7-day challenge window, so blob data remains available for the full duration of any dispute.
- **Permanent commitment.** Even after pruning, a KZG polynomial commitment to the blob is stored on L1 indefinitely, anchoring data availability proofs for future protocols.

Both Arbitrum and Optimism now publish L2 transaction data primarily as blobs rather than calldata. User fees on L2 fell by roughly an order of magnitude following EIP-4844 activation.

Our implementation stores delta arrays in contract storage (`batchDeltas[batchId]`) rather than blobs—a deliberate simplification given our tiny (four-integer) batches. In production the challenger reads from L1 calldata or blob; in our system `getBatchDeltas(batchId)` serves this role. The conceptual point stands: data availability is what makes the challenger's job possible.

8. Optimistic vs. Zero-Knowledge Rollups

Dimension	Optimistic	ZK
Validity model	Assumed valid; challenged if not	Proven mathematically per batch
Withdrawal latency	∼7 days	Minutes
EVM compatibility	Easy — runs EVM directly	Hard — requires zkEVM
Per-batch cost	Cheap (proof only on dispute)	Expensive (SNARK every batch)
Security model	Crypto-economic (1-of-N honest)	Cryptographic / mathematical
Operational maturity	Battle-tested (Arbitrum, Optimism)	Catching up rapidly

Table 1: Comparison of optimistic and zero-knowledge rollups.

Optimistic rollups are **crypto-economic**—secure because lying is unprofitable. ZK rollups are **mathematical**—secure because lying is impossible (modulo cryptographic assumptions and circuit

correctness). Optimistic rollups optimize for cheapness and EVM-compatibility, accepting the seven-day delay. ZK rollups optimize for finality and trust-minimization, accepting higher proving costs.

The current ecosystem consensus is that ZK rollups are the long-term destination, but optimistic rollups are the pragmatic present.

9. Implementation: System Overview

The second half of our project was a working interactive optimistic rollup deployed to Base Sepolia testnet. The implementation has three main parts:

1. `InteractiveOptimisticRollup` — a Solidity smart contract that holds bonds, accepts batch submissions, drives the bisection dispute game, performs single-step verification, and tracks claimable balances after resolution.

2. `interactive-sequencer-runner.ts` — a TypeScript script that posts batches, watches for challenges, and submits midpoint and single-step claims as the dispute game advances.

3. `interactive-challenger-runner.ts` — a TypeScript script that polls for new batches, re-executes them locally, initiates challenges on discrepancy, and submits midpoint and single-step claims.

Both runners interact with the contract via `ethers.js` through Hardhat's network connection. Both operate in a polling loop, acting only when on-chain state requires their input. The runners do not communicate directly with each other—all coordination is mediated through the smart contract, which is the design property that lets the system extend from a demo machine to fully decentralized deployment.

9.1 Simplified State Model: Deltas Instead of Merkle Roots

A real optimistic rollup commits to a Merkle root over every account and storage slot, executes arbitrary EVM transactions, and verifies single steps inside an EVM or MIPS emulator. Our project abstracts away this machinery while preserving the *structure* of the dispute game.

Instead of a Merkle root, the rollup state is a single integer. Instead of an EVM transaction, each "transaction" is an integer **delta** added to the current state:

$$s_0 = \text{initialState}, \qquad s_{i+1} = s_i + d_i$$

The final state of the batch is s_n. This is a 1-D, deterministic, easily-verifiable model that demonstrates dispute mechanics without the engineering burden of a real EVM emulator.

In presentation terms: the delta model is a **proxy for Merkle computation**. In a real system, the "midpoint state at index k" would be a Merkle root R_k over the full rollup state after transaction k, derived from the same accumulation but over a richer state space. We made the accumulation visible (integer math) rather than opaque (hash-tree computations) so that the dispute logic is legible during the demo.

10. The `InteractiveOptimisticRollup` Contract

10.1 Core State Variables

- `sequencerBond`, `challengerBond` — required deposits for batch submission and challenge initiation.
- `challengePeriod` — duration of the challenge window in seconds.
- `batches[batchId]` — per-batch struct: `submittedAt`, `initialState`, `claimedFinalState`, `txCount`, `challenged`, `finalized`, `invalidated`.

- `disputes[batchId]` — per-batch dispute struct tracking the bisection range (`start`, `end`, `mid`), both parties' midpoint and single-step claims, and resolution flags (`sequencerWon`, `challengerWon`).
- `claimableBalances[address]` — withdrawable balance after resolution or finalization.

10.2 Key Functions

- `submitBatch(initialState, claimedFinalState, deltas)` — consumes `sequencerBond`; records the batch; starts the challenge timer.
- `initiateChallenge(batchId, challengerFinalState)` — consumes `challengerBond`; opens a dispute over range [`0`..`txCount` − 1].
- `submitSequencerMidpointClaim` / `submitChallengerMidpointClaim` — each party submits their claimed state at the current range midpoint. When both claims are present, the contract advances the range: if midpoints differ, the lower half is disputed; if they match, the upper half is disputed.
- `submitSequencerSingleStepClaim` / `submitChallengerSingleStepClaim` — once the range is a single index, each party submits their claimed post-state. The contract computes `expected = preState + delta` from its stored data and awards the party whose claim matches.
- `finalizeBatch(batchId)` — callable after the challenge period elapses with no dispute; credits the sequencer's bond back.

The contract emits events at every state transition (`BatchSubmitted`, `ChallengeInitiated`, `MidpointClaimSubmitte`
`DisputeBisected`, `SingleStepClaimSubmitted`, `DisputeResolved`, `BatchFinalized`).

10.3 Bisection State Machine

The dispute moves through three phases. From **WindowOpen**, a call to `initiateChallenge` enters **Bisecting**; expiry with no challenge enters **Finalized**. In **Bisecting**, each round where both midpoint claims are submitted halves the range; when the range collapses to one index the machine enters **SingleStep**. Once both single-step claims are submitted the contract resolves to **ChallengerWon** or **SequencerWon** based on which party's claim matches `preState + delta`.

For our demo batches of 4 transactions, the worst case is 2 bisection rounds plus one single-step verification, consistent with $O(\log_2 4) = 2$.

11. Runner Scripts: Sequencer and Challenger

`interactive-sequencer-runner.ts` posts batches, responds to each bisection round by computing midpoint states from its internal path, and submits single-step claims. Three environment variables control fraud injection: `SUBMITTED_DELTAS_CSV` (what is posted on-chain), `ACTUAL_USED_DELTAS_CSV` (what the sequencer uses internally for dispute assertions—differing values simulate payload-substitution fraud), and `CLAIMED_FINAL_STATE` (the reported final; defaults to applying actual deltas). `STEP_MODE=true` pauses before each on-chain action for legible live demos.

`interactive-challenger-runner.ts` polls for new batches, calls `getBatchDeltas` to re-execute the batch locally, and calls `initiateChallenge` on any discrepancy. It then mirrors every bisection round via `submitChallengerMidpointClaim` and `submitChallengerSingleStepClaim`. `CHALLENGER_WRONG_DELTAS` simulates a frivolous challenge (challenger loses and is slashed).

Race conditions between poll reads and writes are handled throughout both runners by catching known revert selectors (`DisputeNotAtSingleStep` = `0xb111df84`, `MidpointAlreadySubmitted` = `0xe34e893d`).

12. Demonstration on Base Sepolia: Walkthrough and On-Chain Results

The full demonstration was deployed and run on the **Base Sepolia** testnet. Deployed contract address:

`0xA0ECD0679E087a4E821776eAA139E4adc265807d`

Contract configuration: `CHALLENGE_PERIOD = 300 s` (5 minutes, shortened from the production 7-day window for the demo); `SEQUENCER_BOND = CHALLENGER_BOND = 0.005 ETH`.

12.1 Demo Scenario: Sequencer Commits Fraud

The sequencer posted honest deltas $[5, -2, 4, 1]$ on-chain but used fraudulent deltas $[5, -2, \mathbf{5}, 1]$ internally (initial state 10; honest final = 18; claimed final = 19). The challenger re-executed from on-chain data, derived 18, and challenged. Table 2 shows the full state paths.

Index	Pre-state	On-chain delta	Honest post	Seq. (fraud) post	Agree?
0	10	+5	15	15	✓
1	15	−2	13	13	✓
2	13	+4 (seq claims +5)	17	**18**	×
3	17/18	+1	18	**19**	×

Table 2: State paths for the fraud scenario. Divergence begins at index 2.

12.2 Transaction Sequence and Gas Costs

Eight transactions settled the dispute across blocks 41136677–41136693 (16 blocks ≈ 32 s). Table 3 lists each transaction; full hashes follow.

Tx	Function	Block	Gas	Phase total
1	`submitBatch`	41136677	273,306	—
2	`initiateChallenge`	41136679	158,322	—
3	`submitSequencerMidpointClaim`	41136682	101,205	
4	`submitChallengerMidpointClaim`	41136684	70,873	172,078 (R1)
5	`submitSequencerMidpointClaim`	41136686	84,105	
6	`submitChallengerMidpointClaim`	41136688	57,154	141,259 (R2)
7	`submitSequencerSingleStepClaim`	41136691	79,237	
8	`submitChallengerSingleStepClaim`	41136693	110,039	189,276 (SS)
	Total		**934,241**	

Table 3: Transaction sequence, Base Sepolia blocks 41136677–41136693.

Full transaction hashes:

Tx 1: `0x10dd9a73298db741a7098aa8ec4be64a2cf590f2771e7e15fdfc44ed74b54015`

Tx 2: `0xd9087c186e1e1ce802fb801adcd36a0f4b0120d663e70cb1c848690eea819e93`

Tx 3: `0xb762c1a72c87183a4595ab1f8d228f76dc6544f78ea15ba9a04eed4fa329990e`

Tx 4: `0x972f75af1bcff9fd1a0acb830eb02dea94ac35160969be14882bdbf9b47a3c9b`

Tx 5: `0xd357df80d9bff6d12b2abcc69481f41d5264be650a092e001858f80b592e46b8`

Tx 6: `0x609959182c6b0b41c5386da7f648c21b3aab2987c8b2ac1c71d69f862cd6109d`

Tx 7: `0x5abcf661d1ab0f392638597629cfdc6a90b25e5738eb8a01f8f361cff97a52ab`

Tx 8: `0xf425df21f6aedd301cf40326ef90a46efeccce36177d40dbde3fe9855ff42fc2`

Bisection Round 1 (range $[0, 3]$, midpoint 1): both parties claimed state 13 at index 1—agreement. Contract advanced range to $[2, 3]$, emitted `DisputeBisected`.

Bisection Round 2 (range $[2, 3]$, midpoint 2): sequencer claimed 18, challenger claimed 17—disagreement. Contract narrowed to $[2, 2]$.

Single-Step (index 2): sequencer claimed post-state 18; challenger claimed 17. Contract computed $\texttt{expected} = \texttt{preState} + \texttt{batchDeltas}[2] = 13 + 4 = 17$. Challenger matches; **challenger wins**. Batch invalidated; 0.010 ETH (both bonds) credited to challenger.

12.3 Summary of Results

Metric	**Value**
Network	Base Sepolia
Contract address	`0xA0ECD0679E087a4E821776eAA139E4adc265807d`
Batch size	4 transactions
Bisection rounds	2
Total on-chain transactions	8
Tx 1: `submitBatch`	273,306 gas
Tx 2: `initiateChallenge`	158,322 gas
Round 1 bisection (2 txs)	172,078 gas
Round 2 bisection (2 txs)	141,259 gas
Single-step verification (2 txs)	189,276 gas
Total gas used	**934,241**
Block span (Tx 1 to Tx 8)	16 blocks ($\approx$ 32 seconds)
Challenge period (demo config)	300 s (5 minutes)
Dispute outcome	Challenger won; sequencer's bond slashed

Table 4: On-chain demonstration results — Base Sepolia, blocks 41136677–41136693.

The bisection game resolved the 4-transaction dispute in exactly 2 rounds—consistent with the $O(\log_2 4) = 2$ theoretical prediction. L1 re-executed exactly *one* step (index 2) to determine the winner, without ever re-executing the rest of the batch. The entire on-chain dispute settled in approximately 32 seconds of wall-clock time.

13. Discussion: Trade-offs of the Implementation

13.1 Delta Model vs. Real State—and What Merkle Commitment Would Require

The most consequential simplification is the integer-delta state model.

What this preserves: the *shape* of the dispute game—bonds, batch posting, bisection, single-step verification, slashing, finalization. Everything in our contract has a structural counterpart in production rollups.

What this discards: the *content* of execution. We cannot run smart contracts, host applications, or represent multiple accounts. Our "state" is one integer; our "transactions" are integers added to it.

What Merkle-committed state would require differently. If state were committed via a Merkle root rather than a plain integer, the bisection mechanics would need to change in two specific places.

At each midpoint claim: instead of posting a plain integer (e.g., `sequencerStateAtMid = 13`), each party would post a **32-byte state root hash**—a Merkle root over the entire rollup state after executing transactions through that midpoint index. The contract would store and compare these hashes. The comparison semantics are identical (`sequencerMidRoot != challengerMidRoot` still indicates the lower half is disputed), but the claimed value now represents a full EVM state commitment. The parties are bound to this commitment without needing to prove its correctness on-chain at claim time; they cannot later contradict their earlier claim.

At the single-step verification: once the range is a single index i, the single-step resolver must verify three things on-chain rather than one:

1. **Pre-state witness**: a Merkle proof showing that the agreed-upon state before index i—the hash accepted by both parties in the final bisection round—correctly encodes the specific account balances and storage slots that transaction i reads. This *witness* authenticates the pre-state for the disputed step.

2. **Execution**: apply transaction i to the authenticated pre-state. In a production rollup this is a full EVM opcode (Arbitrum) or a MIPS instruction (Optimism's Cannon); in our system it is `preState + delta[i]`.

3. **Post-state commitment**: verify that the resulting post-state, when re-committed into a new Merkle root, produces the hash that one party claimed as their state at index $i + 1$.

The witness construction step (point 1) is the most technically demanding addition. It requires the challenger to submit a Merkle proof showing that the agreed pre-state root at index i contains the specific values the disputed transaction reads. In Optimism's Cannon, this is a MIPS memory witness proving the program state at that single instruction. In Arbitrum, it is an EVM execution proof over the disputed opcode.

Our integer-delta model sidesteps witness construction entirely: the pre-state is a single integer computable on-chain by iterating `batchDeltas[batchId]`, and the "transaction" is integer addition with no external data dependencies. This is why our single-step resolver is trivially cheap (one loop plus one addition) compared to a production dispute resolver that must accept, validate, and apply Merkle witnesses.

13.2 What We Did Right

Despite the simplifications, the implementation faithfully demonstrates several non-obvious properties: **logarithmic bisection** (4-tx batches resolve in 2 rounds + 1 single-step, exactly as predicted); **contract as sole truth** (neither runner can win by lying—the contract re-executes the disputed step against its own stored data); **race conditions are real** (our runners catch `DisputeNotAtSingleStep` and `MidpointAlreadySubmitted` reverts that a naïve implementation would crash on); and **real funds on a real network** (every action required a live L2 transaction).

14. Future Work

Natural next steps, in increasing order of effort:

1. **Replace the delta model with Merkle-rooted state.** State becomes a sparse Merkle tree; transactions become `(from, to, amount)` triples; single-step verification requires Merkle witness

infrastructure as described in Section 13.

2. **Add forced inclusion and a real EVM model.** Minimal EVM interpreter; Patricia trie state commitment; L1 bypass path for censorship resistance.

3. **Combine with a validity proof.** Use a SNARK for the optimistic majority of batches and fall back to fraud proofs for edge cases—the frontier of optimistic-rollup design in 2026.

15. Conclusion

Optimistic rollups are the dominant production answer to Ethereum's scaling problem. They achieve their throughput by relocating execution off-chain while inheriting L1 security through data availability and interactive fraud proofs. Their seven-day challenge window is a calibrated compromise between fraud-detection needs and withdrawal-latency user experience. The bisection game makes single-step verification cost independent of batch size—the property without which fraud proofs would be operationally infeasible.

The architectural difference between Arbitrum and Optimism—BoLD versus Cannon, EVM-direct versus MIPS-emulator—is real but represents different points on a single design axis, not fundamentally different security models. Both reduce a dispute to one execution step on L1; both rely on the same economic argument that lying is unprofitable with a properly-sized bond; both have demonstrated in practice that this argument holds.

Our implementation, while deliberately simplified, exhibits the full structure of an interactive optimistic rollup: bonded sequencer, permissionless challenger, on-chain state machine, and runner agents driving the dispute from both sides. Deployed to Base Sepolia testnet at address `0xA0ECD0679E087a4E821776eAA139E4adc265807d`, the system demonstrated each phase end-to-end in 8 on-chain transactions consuming 934,241 gas total—exactly 2 bisection rounds plus a single-step verification for a 4-transaction batch, matching $O(\log N)$ complexity. The contract re-executed exactly one step, index 2, to determine the winner in 16 blocks (≈32 seconds). It is, at small scale, the security model of Arbitrum or Optimism—proof that the surrounding theory is implementable from first principles in a few hundred lines of code, given the right abstractions.

The single most important insight from the implementation is the inseparability of contract logic and incentive design. The contract does not merely enforce rules—it engineers payoffs. The protocol functions because dishonesty is expensive. Writing the contract is, fundamentally, writing game theory.